



(1) What is Divide and Conquer Technique? Give the use of it for Binary Searching Method.

OR

Explain the use of Divide and Conquer Technique for Binary Search Method. Give the algorithm for Binary Search Method. What is the complexity of Binary Search Method?

Divide and conquer technique.

- Many useful algorithms are **recursive** in structure: to solve a given problem, they call themselves recursively one or more times to deal with closely related sub problems.
- These algorithms typically follow a **divide-and-conquer** approach: they break the problem into several sub problems that are similar to the original problem but smaller in size, solve the sub problems recursively, and then combine these solutions to create a solution to the original problem.
- The divide-and-conquer paradigm involves three steps at each level of the recursion:
 - **Divide** the problem into a number of sub problems.
 - **Conquer** the sub problems by solving them recursively. If the sub problem sizes are small enough, however, just solve the sub problems in a straightforward manner.
 - **Combine** the solutions to the sub problems into the solution for the original problem.

Binary Search Method

- Binary Search is an extremely well-known instance of divide-and-conquer paradigm.
- Let $T[1 \dots n]$ be an array of non-decreasing sorted order; that is $T[i] \leq T[j]$ whenever $1 \leq i \leq j \leq n$.
- Let x be some number. The problem consists of finding x in the array T if it is there.
- If x is not in the array, then we want to find the position where it might be inserted.

Sequential (Linear) search algorithm

Function sequential ($T[1, \dots, n], x$)

```
for i = 1 to n do
    if  $T[i] \geq x$  then return index i
return n + 1
```

Analysis

- Here we look sequentially at each element of T until either we reach to end of array or find a number no smaller than x .
- This algorithm clearly takes time in $\Theta(r)$, where r is the index returned. $\Theta(n)$ in worst case and $O(1)$ in best case.

Binary Search Algorithm (Iterative)

- The basic idea of binary search is that for a given element we check out the middle element of the array. We continue in either the lower or upper segment of the array, depending on the outcome of the search until we reached the required (given) element.



Function binitier (T[1,...,n], x)

```

i ← 1; j ← n
while i < j do
    k ← (i + j) ÷ 2
    case x < T[k] : j ← k - 1
        x = T[k] : i, j ← k {return k}
    x > T[k] : i ← k + 1
return i

```

Analysis

- To analyze the running time of a while loop, we must find a function of the variables involved whose value decreases each time round the loop.
- Here it is $j - i + 1$
- Which we call as d. d represents the number of elements of T still under consideration.
- Initially $d = n$.
- Loop terminates when $i \geq j$, which is equivalent to $d \leq 1$
- Each time round the loop there are three possibilities,
 - Either j is set to $k - 1$
 - Or i is set to $k + 1$
 - Or both i and j are set to k
- Let d and d' stand for the value of $j - i + 1$ before and after the iteration under consideration. Similarly i, j, i' and j'.
- Case I : if $x < T[k]$

So, $j \leftarrow k - 1$ is executed.

Thus $i' = i$ and $j' = k - 1$ where $k = (i + j) \div 2$

Substituting the value of k, $j' = [(i + j) \div 2] - 1$

$d' = j' - i' + 1$

Substituting the value of j', $d' = [(i + j) \div 2] - 1 - i + 1$

$d' \leq (i + j) / 2 - i$

$d' \leq (j - i) / 2 \leq (j - i + 1) / 2$

$d' \leq d/2$
- Case II : if $x > T[k]$

So, $i \leftarrow k + 1$ is executed

Thus $i' = k + 1$ and $j' = j$ where $k = (i + j) \div 2$

Substituting the value of k, $i' = [(i + j) \div 2] + 1$

$d' = j' - i' + 1$

Substituting the value of i', $d' = j - [(i + j) \div 2] + 1 + 1$

$d' \leq j - (i + j - 1) / 2 \leq (2j - i - j + 1) / 2 \leq (j - i + 1) / 2$

$d' \leq d/2$



- Case III : if $x = T[k]$
 $i = j \rightarrow d' = 1$
- We conclude that whatever may be case, $d' \leq d/2$ which means that the value of d is at least getting half each time round the loop.
- Let d_k denote the value of $j - i + 1$ at the end of k^{th} trip around the loop. $d_0 = n$.
- We have already proved that $d_k = d_{k-1} / 2$
- For n integers, how many times does it need to cut in half before it reaches or goes below 1?
- $n / 2^k \leq 1 \rightarrow n \leq 2^k$
- $k = \lg n$, search takes time.
- **The complexity of biniter is $\Theta(\lg n)$.**

Binary Search Algorithm (Recursive)

Function binsearch (T[1,...,n], x)

if $n = 0$ **or** $x > T[n]$ **then return** $n + 1$

else return binrec (T[1,...,n], x)

Function binrec(T[i,...,j], x)

if $i = j$ **then return** i

$k \leftarrow (i + j) \div 2$

if $x \leq T[k]$ **then return** binrec(T[i,...,k], x)

else return binrec(T[k + 1,...,j], x)

Analysis

- Let $t(n)$ be the time required for a call on binrec(T[i,...,j], x), where $n = j - i + 1$ is the number of elements still under consideration in the search.
- The recurrence equation is given as,
 $t(n) = t(n/2) + \Theta(1)$
 Comparing this to the general template for divide and conquer algorithm, $l = 1$, $b = 2$ and $k = 0$.
 So, $t(n) \in \Theta(\lg n)$
- **The complexity of binrec is $\Theta(\lg n)$.**

(2) Write an algorithm / method for Merge Sort. Analyze the algorithm and find its time complexity.

- The divide & conquer approach to sort n numbers using merge sort consists of separating the array T into two parts where sizes are almost same.
- These two parts are sorted by recursive calls and then merged the solution of each part while preserving the order. The algorithm considers two temporary arrays U and V into which the original array T is divided.
- When the number of elements to be sorted is small, a relatively simple algorithm is used.
- Merge sort procedure separates the instance into two half sized sub instances, solves them recursively and then combines the two sorted half arrays to obtain the solution to the original instance.



Algorithm for merging two sorted U and V arrays into array T

Procedure merge($U[1, \dots, m+1], V[1, \dots, n+1], T[1, \dots, m+n]$)

```
i, j  $\leftarrow$  1
U[m+1], V[n+1]  $\leftarrow \infty$ 
for k  $\leftarrow$  1 to m + n do
    if U[i] < V[j]
        then T[k]  $\leftarrow$  U[i] ; i  $\leftarrow$  i + 1
        else T[k]  $\leftarrow$  V[j] ; j  $\leftarrow$  j + 1
```

Algorithm merge sort

Procedure mergesort($T[1, \dots, n]$)

```
if n is sufficiently small then insert(T)
else
    array U[1, ..., 1+n/2], V[1, ..., 1+n/2]
    U[1, ..., n/2]  $\leftarrow$  T[1, ..., n/2]
    V[1, ..., n/2]  $\leftarrow$  T[n/2+1, ..., n]
    mergesort(U[1, ..., n/2])
    mergesort(V[1, ..., n/2])
    merge(U, V, T)
```

Analysis

- Let $T(n)$ be the time taken by this algorithm to sort an array of n elements.
- Separating T into U & V takes linear time; merge (U, V, T) also takes linear time.
- Now,

$$T(n) = T(n/2) + T(n/2) + g(n) \quad \text{where } g(n) \in \Theta(n).$$

$$T(n) = 2T(n/2) + \Theta(n)$$

Applying the general case,

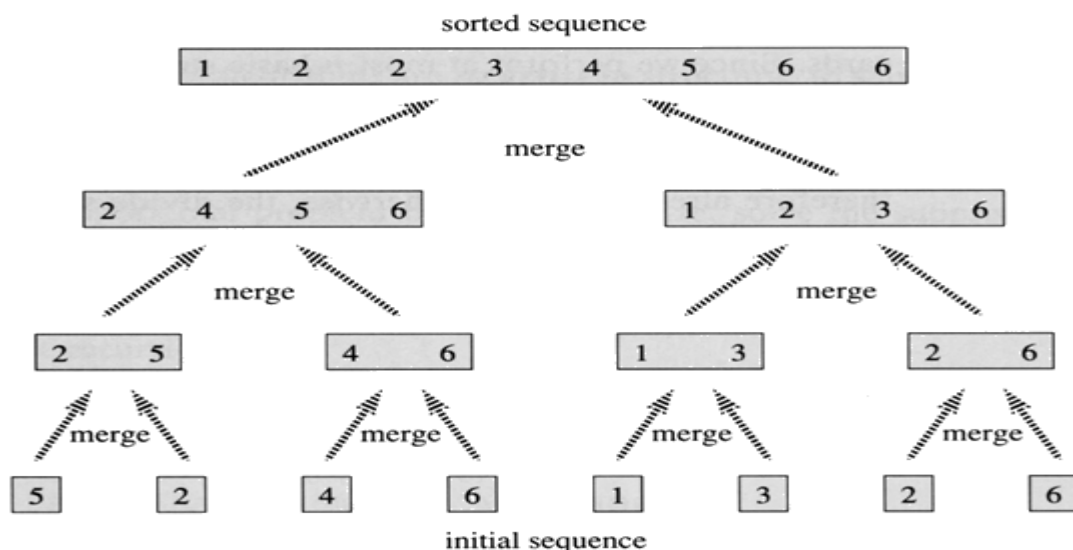
$$l=2, b=2, k=1$$

Since $l = b^k$ the second case applies which yields $t(n) \in \Theta(n \log n)$.

Time complexity of merge sort is $\Theta(n \log n)$.



Example



(3) Explain Quick sort method / algorithm and find its complexity.

- Quick sort works by partitioning the array to be sorted.
- Each Partition is internally sorted recursively.
- As a first step, this algorithm chooses one element of an array as a pivot or a key element.
- The array is then partitioned on either side of the pivot.
- Elements are moved so that those greater than the pivot are shifted to its right whereas the others are shifted to its left.
- Two pointers low and up are initialized to the lower and upper bounds of the sub array.
- Up pointer will be decremented and low pointer will be incremented as per following condition.
 1. Increase low pointer until $T[\text{low}] > \text{pivot}$.
 2. Decrease up pointer until $T[\text{up}] \leq \text{pivot}$.
 3. If $\text{low} < \text{up}$ then interchange $T[\text{low}]$ with $T[\text{up}]$.
 4. If $\text{up} < \text{low}$ then interchange $T[\text{up}]$ with $T[\text{low}]$.

Algorithm

Procedure pivot($T[i, \dots, j]$; var l)

{Permutates the elements in array $T[i, \dots, j]$ and returns a value l such that, at the end, $i \leq l \leq j$, $T[k] \leq p$ for all $i \leq k < l$, $T[l] = p$, And $T[k] > p$ for all $l < k \leq j$, where p is the initial value $T[i]$ }

$P \leftarrow T[i]$

$K \leftarrow i$; $l \leftarrow j+1$

Repeat $k \leftarrow k+1$ **until** $T[k] > p$

Repeat $l \leftarrow l-1$ **until** $T[l] \leq p$

While $k < l$ **do**

 Swap $T[k]$ and $T[l]$



Repeat $k \leftarrow k+1$ **until** $T[k] > p$
Repeat $l \leftarrow l-1$ **until** $T[l] \leq p$
 Swap $T[i]$ and $T[l]$

Procedure quicksort($T[i, \dots, j]$)
 {Sorts subarray $T[i, \dots, j]$ into non decreasing order}
if $j - i$ is sufficiently small **then** insert ($T[i, \dots, j]$)
else
 pivot($T[i, \dots, j], l$)
 quicksort($T[i, \dots, l - 1]$)
 quicksort($T[l+1, \dots, j]$)

Analysis

1. Worst Case

- Running time of quick sort depends on whether the partitioning is balanced or unbalanced.
- And this in turn depends on which element is chosen as key or pivot element.
- The worst case behavior for quick sort occurs when the partitioning routine produces one sub problem with $n-1$ elements and one with 0 elements.
- In this case recurrence will be,

$$T(n) = T(n-1) + T(0) + \Theta(n)$$

$$T(n) = T(n-1) + \Theta(n)$$

$$T(n) = \Theta(n^2)$$

2. Best Case

- Occurs when partition produces sub problems each of size $n/2$.
- Recurrence equation:

$$T(n) = 2T(n/2) + \Theta(n)$$

$$l = 2, b = 2, k = 1, \text{ so } l = b^k$$

$$T(n) = \Theta(n \log n)$$

3. Average Case

- Average case running time is much closer to the best case.
- If suppose the partitioning algorithm produces a 9-to-1 proportional split the recurrence will be

$$T(n) = T(9n/10) + T(n/10) + \Theta(n)$$
 Solving it,

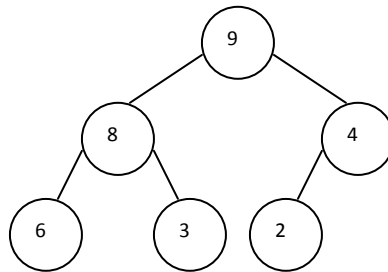
$$T(n) = \Theta(n \log n)$$
- The running time of quick sort is therefore $\Theta(n \log n)$ whenever the split has constant proportionality.**



(4) Heaps

- A heap data structure is a binary tree with the following properties.
 1. It is a complete binary tree; that is each level of the tree is completely filled, except possibly the bottom level. At this level it is filled from left to right.
 2. It satisfies the heap order property; the data item stored in each node is greater than or equal to the data item stored in its children node.

Example:



- Heap can be implemented using an array or a linked list structure. It is easier to implement heaps using arrays.
- We simply number the nodes in the heap from top to bottom, numbering the nodes on each level from left to right and store the i^{th} node in the i^{th} location of the array.
- An array A that represents a heap is an object with two attributes:
 - i. $length[A]$, which is the number of elements in the array, and
 - ii. $heap-size[A]$, the number of elements in the heap stored within array A .
- The root of the tree is $A[1]$, and given the index i of a node, the indices of its parent $PARENT(i)$, left child $LEFT(i)$, and right child $RIGHT(i)$ can be computed simply:

$PARENT(i)$

return $\lfloor i/2 \rfloor$

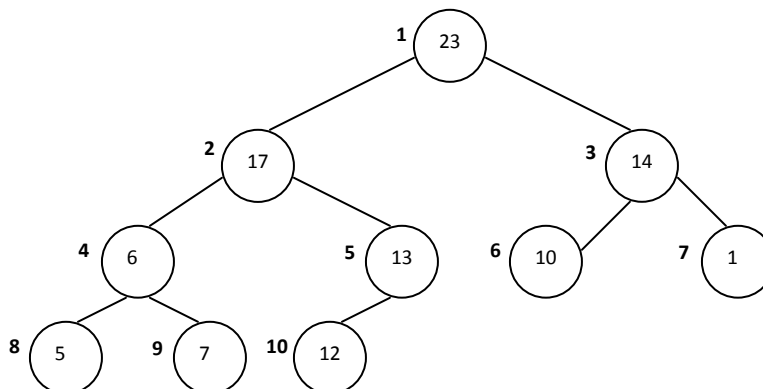
$LEFT(i)$

return $2i$

$RIGHT(i)$

return $2i + 1$

Example:





- The array form for the above heap is,

23	17	14	6	13	10	1	5	7	12
----	----	----	---	----	----	---	---	---	----

- There are two kinds of binary heaps: max-heaps and min-heaps. In both kinds, the values in the nodes satisfy a **heap property**.
- In a **max-heap**, the **max-heap property** is that for every node i other than the root, $A[\text{PARENT}(i)] \geq A[i]$,
- That is, the value of a node is at most the value of its parent. Thus, the largest element in a max-heap is stored at the root, and the sub-tree rooted at a node contains values no larger than that contained at the node itself.
- A **min-heap** is organized in the opposite way; the **min-heap property** is that for every node i other than the root, $A[\text{PARENT}(i)] \leq A[i]$.
- The smallest element in a min-heap is at the root.
- For the heap-sort algorithm, we use max-heaps. Min-heaps are commonly used in priority Queues.
- Viewing a heap as a tree, we define the **height** of a node in a heap to be the number of edges on the longest simple downward path from the node to a leaf, and we define the height of the Heap to be the height of its root.
- Height of an n element heap based on a binary tree is $\lg n$
- The basic operations on heap run in time at most proportional to the height of the tree and thus take $O(\lg n)$ time.
- Since a heap of n elements is a complete binary tree which has height k ; that is one node on level k , two nodes on level $k-1$ and so on...
- There will be 2^{k-1} nodes on level 1 and at least 1 and not more than 2^k nodes on level 0.

Building a heap

- For the general case of converting a complete binary tree to a heap, we begin at the last node that is not a leaf; apply the “percolate down” routine to convert the subtree rooted at this current root node to a heap.
- We then move onto the preceding node and percolate down that subtree.
- We continue on in this manner, working up the tree until we reach the root of the given tree.
- We can use the procedure MAX-HEAPIFY in a bottom-up manner to convert an array $A[1, \dots, n]$, where $n = \text{length}[A]$, into a max-heap.
- The elements in the sub-array $A[(\lfloor n/2 \rfloor + 1), \dots, n]$ are all leaves of the tree, and so each is a 1-element heap to begin with.
- The procedure BUILD-MAX-HEAP goes through the remaining nodes of the tree and runs MAXHEAPIFY on each one.



Algorithm

```
BUILD-MAX-HEAP(A)
heap-size[A] ← length[A]
for i ← ⌊length[A]/2⌋ downto 1
    do MAX-HEAPIFY(A, i)
```

Analysis

- Each call to Heapify costs $O(\lg n)$ time, and there are $O(n)$ such calls. Thus, the running time is at most $O(n \lg n)$

Maintaining the heap property

- One of the most basic heap operations is converting a complete binary tree to a heap. Such an operation is called Heapify.
- Its inputs are an array A and an index i into the array. When MAX-HEAPIFY is called, it is assumed that the binary trees rooted at LEFT(i) and RIGHT(i) are max-heaps, but that $A[i]$ may be smaller than its children, thus violating the max-heap property.
- The function of MAX-HEAPIFY is to let the value at $A[i]$ "float down" in the max-heap so that the sub-tree rooted at index i becomes a max-heap.

Algorithm

```
MAX-HEAPIFY(A, i)
l ← LEFT(i)
r ← RIGHT(i)
if l ≤ heap-size[A] and A[l] > A[i]
    then largest ← l
else largest ← i
if r ≤ heap-size[A] and A[r] > A[largest]
    then largest ← r
if largest ≠ i
    then exchange A[i] ↔ A[largest]
MAX-HEAPIFY(A, largest)
```

- At each step, the largest of the elements $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$ is determined, and its index is stored in *largest*.
- If $A[i]$ is largest, then the sub-tree rooted at node i is a max-heap and the procedure terminates.
- Otherwise, one of the two children has the largest element, and $A[i]$ is swapped with $A[\text{largest}]$, which causes node i and its children to satisfy the max-heap property.
- The node indexed by *largest*, however, now has the original value $A[i]$, and thus the sub-tree rooted at *largest* may violate the max-heap property. therefore, MAX-HEAPIFY must be called recursively on that sub-tree.



Analysis

- The running time of MAX-HEAPIFY on a sub-tree of size n rooted at given node i is the $\Theta(1)$ time to fix up the relationships among the elements $A[i]$, $A[\text{LEFT}(i)]$, and $A[\text{RIGHT}(i)]$, plus the time to run MAX-HEAPIFY on a sub-tree rooted at one of the children of node i .
- The children's sub-trees can have size of at most $2n/3$ and the running time of MAX-HEAPIFY can therefore be described by the recurrence

$$T(n) \leq T(2n/3) + \Theta(1).$$

- The solution to this recurrence is $T(n) = O(\lg n)$.

The heapsort algorithm

- The heapsort algorithm starts by using BUILD-MAX-HEAP to build a max-heap on the input array $A[1, \dots, n]$, where $n = \text{length}[A]$.
- Since the maximum element of the array is stored at the root $A[1]$, it can be put into its correct final position by exchanging it with $A[n]$.
- If we now "discard" node n from the heap (by decrementing $\text{heap-size}[A]$), we observe that $A[1, \dots, (n - 1)]$ can easily be made into a max-heap.
- The children of the root remain max-heaps, but the new root element may violate the max-heap property.
- All that is needed to restore the max heap property, however, is one call to MAX-HEAPIFY($A, 1$), which leaves a max-heap in $A[1, \dots, (n - 1)]$.
- The heapsort algorithm then repeats this process for the max-heap of size $n - 1$ down to a heap of size 2.

Algorithm

HEAPSORT(A)

 BUILD-MAX-HEAP(A)

 for $i \leftarrow \text{length}[A]$ downto 2

 do exchange $A[1] \leftrightarrow A[i]$

$\text{heap-size}[A] \leftarrow \text{heap-size}[A] - 1$

 MAX-HEAPIFY($A, 1$)

The HEAPSORT procedure takes time $O(n \lg n)$, since the call to BUILD-MAX-HEAP takes time $O(n)$ and each of the $n - 1$ calls to MAX-HEAPIFY takes time $O(\lg n)$.